

Infosys Most Asked C++ DSA Questions

(Arrays, Strings, Linked List, Stack, Queue, Trees, BST etc)

Abhishek Rathor

Instagram id: SYNTAX ERROR (code.abhii07)

Key Features:

- 75+ Interview Questions from coding rounds.
- Beginner to Advanced Topics: Arrays → Strings → Linked List → Stack → Queue → Trees → BST → Advanced.
- Code in C++ with Time & Space Complexity.
- Concept Explanations included for quick revision.
- Optimized Solutions for interviews and campus placements.

Why This Guide?

- **Save Time:** No need to search multiple sources.
- **Practice Smart:** Focus on most frequently asked questions.
- **Learn Concepts:** Step-by-step explanations for each topic.
- **Placement Ready:** Covers all rounds and coding challenges.

Success is not just about writing code, it's about solving problems.

Arrays

1. LeetCode: 493 (Reverse Pairs) — related to count inversions

Question:

Count the number of inversions in the array, i.e., pairs (i, j) such that $i < j$ and $arr[i] > arr[j]$.

Solution Logic:

Use modified merge sort: during merge, when right element is smaller, count inversions equal to remaining left elements.

Code (C++):

```
#include <vector>
using namespace std;

long long merge_count(vector<int>& arr, int l, int m, int r) {
    vector<int> left(arr.begin()+l, arr.begin()+m+1);
    vector<int> right(arr.begin()+m+1, arr.begin()+r+1);
    int i = 0, j = 0, k = l;
    long long inv = 0;
    while (i < left.size() && j < right.size()) {
        if (left[i] <= right[j]) {
            arr[k++] = left[i++];
        } else {
            arr[k++] = right[j++];
            inv += (left.size() - i);
        }
    }
    while (i < left.size()) arr[k++] = left[i++];
    while (j < right.size()) arr[k++] = right[j++];
    return inv;
}

long long mergeSortCount(vector<int>& arr, int l, int r) {
    if (l >= r) return 0;
    int m = l + (r - l) / 2;
    long long inv = mergeSortCount(arr, l, m) + mergeSortCount(arr, m+1, r);
    inv += merge_count(arr, l, m, r);
    return inv;
}

long long countInversions(vector<int>& arr) {
    return mergeSortCount(arr, 0, arr.size()-1);
}
```

Time Complexity: $O(n \log n)$

Space Complexity: $O(n)$

2. LeetCode: — (no exact match)

Question:

Given an integer array and a target sum K , determine if there exists a pair whose sum equals K .

Solution Logic:

Use a hash set to check for complement $K - x$ while traversing.

Code (C++):

```
#include <vector>
#include <unordered_set>
using namespace std;

bool hasPairWithSum(const vector<int>& arr, int K) {
    unordered_set<int> seen;
    for (int x : arr) {
        if (seen.count(K - x)) {
            return true;
        }
        seen.insert(x);
    }
    return false;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

3. LeetCode: — (no exact match)

Question:

Given an integer array, find the largest and smallest elements present in the array.

Solution Logic:

Scan once keeping track of min and max seen so far.

Code (C++):

```
#include <vector>
#include <limits>
using namespace std;

pair<int,int> findMinMax(const vector<int>& arr) {
    int mn = numeric_limits<int>::max();
    int mx = numeric_limits<int>::min();
    for (int x : arr) {
        if (x < mn) mn = x;
        if (x > mx) mx = x;
    }
    return {mn, mx};
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

4. LeetCode: 189 (Rotate Array)

Question:

Rotate an array of n elements to the right by k steps.

Solution Logic:

Normalize k , reverse the entire array, then reverse first k and then the rest.

Code (C++):

```
#include <vector>
#include <algorithm>
using namespace std;

void reverseRange(vector<int>& arr, int l, int r) {
    while (l < r) {
        swap(arr[l++], arr[r--]);
    }
}

void rotateArray(vector<int>& arr, int k) {
    int n = arr.size();
    k %= n;
    reverseRange(arr, 0, n - 1);
    reverseRange(arr, 0, k - 1);
    reverseRange(arr, k, n - 1);
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

5. LeetCode: 283 (Move Zeroes)

Question:

Move all zeroes in the array to the end, maintaining the order of non-zero elements.

Solution Logic:

Two-pointer approach: place non-zero at front, then fill remainder with zero.

Code (C++):

```
#include <vector>
using namespace std;

void moveZeroes(vector<int>& arr) {
    int pos = 0;
    for (int x : arr) {
        if (x != 0) {
            arr[pos++] = x;
        }
    }
    while (pos < arr.size()) {
        arr[pos++] = 0;
    }
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

6. LeetCode: — (no direct match)

Question:

Given two sorted arrays, merge them into one sorted array.

Solution Logic:

Use two pointers to pick smaller from both arrays, merge into result.

Code (C++):

```
#include <vector>
using namespace std;

vector<int> mergeSorted(const vector<int>& a, const vector<int>& b) {
    vector<int> result;
    int i = 0, j = 0;
    while (i < a.size() && j < b.size()) {
        if (a[i] < b[j]) result.push_back(a[i++]);
        else result.push_back(b[j++]);
    }
    while (i < a.size()) result.push_back(a[i++]);
    while (j < b.size()) result.push_back(b[j++]);
    return result;
}
```

Time Complexity: $O(n + m)$

Space Complexity: $O(n + m)$

7. LeetCode: 169 (Majority Element)

Question:

Find the element that appears more than $\lfloor n/2 \rfloor$ times in the array.

Solution Logic:

Use Boyer–Moore voting algorithm to find candidate, then confirm.

Code (C++):

```
#include <vector>
using namespace std;

int majorityElement(const vector<int>& nums) {
    int count = 0;
    int candidate = 0;
    for (int x : nums) {
        if (count == 0) candidate = x;
        count += (x == candidate) ? 1 : -1;
    }
    // Optional verification step if needed
```

```
    return candidate;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

8. LeetCode: 53 (Maximum Subarray)

Question:

Find the contiguous subarray (containing at least one number) which has the largest sum.

Solution Logic:

Kadane's algorithm: maintain current sum and max sum, reset when negative.

Code (C++):

```
#include <vector>
#include <algorithm>
using namespace std;

int maxSubArray(const vector<int>& nums) {
    int current = nums[0], best = nums[0];
    for (int i = 1; i < nums.size(); i++) {
        current = max(nums[i], current + nums[i]);
        best = max(best, current);
    }
    return best;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

9. LeetCode: 1 (Two Sum)

Question:

Given an array and a target, return indices of the two numbers such that they add up to target.

Solution Logic:

Use a hash map (`unordered_map`) to store numbers and their indices; check complement.

Code (C++):

```
#include <vector>
#include <unordered_map>
using namespace std;

vector<int> twoSum(const vector<int>& nums, int target) {
    unordered_map<int, int> mp;
    for (int i = 0; i < nums.size(); i++) {
        int comp = target - nums[i];
        if (mp.count(comp)) {
            return {mp[comp], i};
        }
    }
}
```

```

    }
    mp[nums[i]] = i;
}
return {};
}

```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

10. LeetCode: 259 (3Sum Smaller) — similar pattern

Question:

Count number of triplets in the array with sum less than X.

Solution Logic:

Sort the array, then fix one element and use two pointers for the rest.

Code (C++):

```

#include <vector>
#include <algorithm>
using namespace std;

int threeSumSmaller(vector<int>& nums, int target) {
    sort(nums.begin(), nums.end());
    int count = 0;
    for (int i = 0; i < nums.size() - 2; i++) {
        int left = i + 1, right = nums.size() - 1;
        while (left < right) {
            if (nums[i] + nums[left] + nums[right] < target) {
                count += (right - left);
                left++;
            } else {
                right--;
            }
        }
    }
    return count;
}

```

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

11. LeetCode: 560 (Subarray Sum Equals K) — positive only variant

Question:

Given an array of positive integers and a target, find the start and end indices of a subarray whose sum equals target.

Solution Logic:

Use sliding window: expand right, shrink left when sum exceeds target.

Code (C++):

```
#include <vector>
using namespace std;

pair<int,int> subarraySum(const vector<int>& nums, int target) {
    int start = 0;
    int sum = 0;
    for (int end = 0; end < nums.size(); end++) {
        sum += nums[end];
        while (sum > target && start <= end) {
            sum -= nums[start++];
        }
        if (sum == target) {
            return {start, end};
        }
    }
    return {-1, -1};
}
```

Time Complexity: $O(n)$ **Space Complexity:** $O(1)$

12. LeetCode: 238 (Product of Array Except Self)

Question:

Return an array output such that output[i] is the product of all elements except nums[i].

Solution Logic:

Compute prefix and suffix products to get result without division.

Code (C++):

```
#include <vector>
using namespace std;

vector<long long> productExceptSelf(const vector<int>& nums) {
    int n = nums.size();
    vector<long long> result(n, 1);
    long long prefix = 1;
    for (int i = 0; i < n; i++) {
        result[i] = prefix;
        prefix *= nums[i];
    }
    long long suffix = 1;
    for (int i = n - 1; i >= 0; i--) {
        result[i] *= suffix;
        suffix *= nums[i];
    }
    return result;
}
```

Time Complexity: $O(n)$ **Space Complexity:** $O(n)$

13. LeetCode: 442 (Find All Duplicates in an Array)

Question:

Find all elements that appear twice in an array of n integers where each integer is between 1 and n .

Solution Logic:

Use index marking by negating value at index to detect duplicates.

Code (C++):

```
#include <vector>
using namespace std;

vector<int> findDuplicates(vector<int>& nums) {
    vector<int> duplicates;
    for (int i = 0; i < nums.size(); i++) {
        int idx = abs(nums[i]) - 1;
        if (nums[idx] < 0) {
            duplicates.push_back(idx + 1);
        } else {
            nums[idx] = -nums[idx];
        }
    }
    return duplicates;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

14. LeetCode: — (similar to Best Time to Buy & Sell Stock)

Question:

Find the maximum difference $\text{arr}[j] - \text{arr}[i]$ such that $j > i$.

Solution Logic:

Track minimum so far and compute potential max differences.

Code (C++):

```
#include <vector>
#include <limits>
using namespace std;

int maxDiff(const vector<int>& arr) {
    int minVal = numeric_limits<int>::max();
    int maxDiff = 0;
    for (int x : arr) {
        if (x - minVal > maxDiff) {
            maxDiff = x - minVal;
        }
        if (x < minVal) {
            minVal = x;
        }
    }
    return maxDiff;
}
```

```
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

15. LeetCode: 918 (Maximum Sum Circular Subarray)

Question:

Find the maximum subarray sum in a circular array.

Solution Logic:

Find max subarray (Kadane), find min subarray, subtract from total to get wrap-around case.

Code (C++):

```
#include <vector>
#include <algorithm>
using namespace std;

int kadane(const vector<int>& nums) {
    int max_ending = nums[0], max_so_far = nums[0];
    for (int i = 1; i < nums.size(); i++) {
        max_ending = max(nums[i], max_ending + nums[i]);
        max_so_far = max(max_so_far, max_ending);
    }
    return max_so_far;
}

int maxSubarrayCircular(const vector<int>& nums) {
    int max_kadane = kadane(nums);
    int total = 0;
    vector<int> inverted(nums.size());
    for (int i = 0; i < nums.size(); i++) {
        total += nums[i];
        inverted[i] = -nums[i];
    }
    int max_inverted = kadane(inverted);
    int max_circular = total + max_inverted;
    if (max_circular == 0) {
        return max_kadane;
    }
    return max(max_kadane, max_circular);
}
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Strings

16. LeetCode: 344 (Reverse String)

Question:

Given a string, reverse it without using built-in reverse functions.

Solution Logic:

Swap characters from two ends moving inwards with two pointers.

Code (C++):

```
#include <string>
using namespace std;

string reverseString(string s) {
    int left = 0, right = s.size() - 1;
    while (left < right) {
        swap(s[left++], s[right--]);
    }
    return s;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

17. LeetCode: 125 (Valid Palindrome)

Question:

Check whether a given string is a palindrome (ignoring non-alphanumeric characters, case insensitive).

Solution Logic:

Use two pointers, skip non-alphanumeric, compare lowercase characters.

Code (C++):

```
#include <string>
using namespace std;

bool isPalindrome(string s) {
    int left = 0, right = s.size() - 1;
    while (left < right) {
        while (left < right && !isalnum(s[left])) left++;
        while (left < right && !isalnum(s[right])) right--;
        if (tolower(s[left]) != tolower(s[right])) return false;
        left++;
        right--;
    }
    return true;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

18. LeetCode: 28 (Implement strStr)

Question:

Find the first occurrence of substring `needle` in `haystack`. Return index or `-1`.

Solution Logic:

Check each starting position using substring comparison (or use KMP for optimized).

Code (C++):

```
#include <string>
using namespace std;

int strStr(string haystack, string needle) {
    if (needle.empty()) return 0;
    int n = haystack.size(), m = needle.size();
    for (int i = 0; i + m <= n; i++) {
        if (haystack.substr(i, m) == needle) {
            return i;
        }
    }
    return -1;
}
```

Time Complexity: $O((n - m) * m)$

Space Complexity: $O(1)$

19. LeetCode: 38 (Count and Say)

Question:

Generate the n -th term of the “count-and-say” sequence.

Solution Logic:

Build term by reading previous term, counting consecutive same characters.

Code (C++):

```
#include <string>
using namespace std;

string countAndSay(int n) {
    string result = "1";
    for (int i = 1; i < n; i++) {
        string temp;
        int count = 1;
        for (int j = 1; j < result.size(); j++) {
            if (result[j] == result[j - 1]) {
                count++;
            } else {
                temp += to_string(count) + result[j - 1];
                count = 1;
            }
        }
    }
}
```

```

    }
    temp += to_string(count) + result.back();
    result = temp;
}
return result;
}

```

Time Complexity: $O(n * m)$ where m grows roughly in sequence length

Space Complexity: $O(m)$

20. LeetCode: 14 (Longest Common Prefix)

Question:

Given an array of strings, find the longest common prefix.

Solution Logic:

Take first string as prefix, shrink it when mismatch with others.

Code (C++):

```

#include <vector>
#include <string>
using namespace std;

string longestCommonPrefix(vector<string>& strs) {
    if (strs.empty()) return "";
    string prefix = strs[0];
    for (int i = 1; i < strs.size(); i++) {
        while (strs[i].find(prefix) != 0) {
            prefix = prefix.substr(0, prefix.size() - 1);
            if (prefix.empty()) return "";
        }
    }
    return prefix;
}

```

Time Complexity: $O(S)$ where S is sum of lengths of all strings

Space Complexity: $O(1)$

21. LeetCode: 242 (Valid Anagram)

Question:

Given two strings s and t , check if t is an anagram of s .

Solution Logic:

Count frequencies of characters and compare or sort and compare.

Code (C++):

```

#include <string>
#include <array>
using namespace std;

```

```

bool isAnagram(string s, string t) {
    if (s.size() != t.size()) return false;
    array<int, 26> freq = {0};
    for (char c : s) freq[c - 'a']++;
    for (char c : t) {
        if (--freq[c - 'a'] < 0) return false;
    }
    return true;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

22. LeetCode: 387 (First Unique Character in a String)

Question:

Find the first non-repeating character's index in a string. Return -1 if none.

Solution Logic:

Count frequencies, then scan to find first with count = 1.

Code (C++):

```

#include <string>
#include <vector>
using namespace std;

int firstUniqChar(string s) {
    vector<int> freq(26, 0);
    for (char c : s) freq[c - 'a']++;
    for (int i = 0; i < s.size(); i++) {
        if (freq[s[i] - 'a'] == 1) return i;
    }
    return -1;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

23. LeetCode: 8 (String to Integer (atoi))

Question:

Convert string to integer (handle sign, overflow, invalid chars).

Solution Logic:

Parse chars, manage sign, clamp to 32-bit limits.

Code (C++):

```

#include <string>
#include <climits>

```

```

using namespace std;

int myAtoi(string s) {
    int i = 0, sign = 1;
    long long result = 0;
    while (i < s.size() && isspace(s[i])) i++;
    if (i < s.size() && (s[i] == '+' || s[i] == '-')) {
        sign = (s[i++] == '-') ? -1 : 1;
    }
    while (i < s.size() && isdigit(s[i])) {
        result = result * 10 + (s[i++] - '0');
        if (result * sign > INT_MAX) return INT_MAX;
        if (result * sign < INT_MIN) return INT_MIN;
    }
    return (int)(result * sign);
}

```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

24. LeetCode: 5 (Longest Palindromic Substring)

Question:

Find the longest palindromic substring in a string.

Solution Logic:

Expand around each character (odd & even centers) and track longest bounds.

Code (C++):

```

#include <string>
using namespace std;

pair<int,int> expandAround(const string& s, int left, int right) {
    while (left >= 0 && right < s.size() && s[left] == s[right]) {
        left--; right++;
    }
    return {left + 1, right - 1};
}

string longestPalindrome(string s) {
    if (s.empty()) return "";
    int start = 0, end = 0;
    for (int i = 0; i < s.size(); i++) {
        auto [l1, r1] = expandAround(s, i, i);
        auto [l2, r2] = expandAround(s, i, i + 1);
        if (r1 - l1 > end - start) {
            start = l1; end = r1;
        }
        if (r2 - l2 > end - start) {
            start = l2; end = r2;
        }
    }
    return s.substr(start, end - start + 1);
}

```

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

25. LeetCode: 151 (Reverse Words in a String)

Question:

Reverse the order of words in a string (trim spaces, clean duplicates).

Solution Logic:

Split words, then combine in reverse order.

Code (C++):

```
#include <sstream>
#include <string>
#include <vector>
using namespace std;

string reverseWords(string s) {
    istringstream iss(s);
    vector<string> words;
    string word;
    while (iss >> word) {
        words.push_back(word);
    }
    string result;
    for (int i = words.size() - 1; i >= 0; i--) {
        result += words[i];
        if (i > 0) result += ' ';
    }
    return result;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

26. LeetCode: 680 (Valid Palindrome II)

Question:

Check if string can become palindrome by removing at most one character.

Solution Logic:

Use two pointers; on mismatch, skip one side and verify remainder is palindrome.

Code (C++):

```
#include <string>
using namespace std;

bool isPalRange(const string& s, int i, int j) {
    while (i < j) {
        if (s[i++] != s[j--]) return false;
    }
    return true;
}
```

```

    }
    return true;
}

bool validPalindrome(string s) {
    int left = 0, right = s.size() - 1;
    while (left < right) {
        if (s[left] != s[right]) {
            return isPalRange(s, left + 1, right) || isPalRange(s, left, right - 1);
        }
        left++; right--;
    }
    return true;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

27. LeetCode: 205 (Isomorphic Strings)

Question:

Determine if two strings are isomorphic (characters map one-to-one).

Solution Logic:

Maintain two mappings ($s \rightarrow t$, $t \rightarrow s$) and ensure consistency.

Code (C++):

```

#include <string>
#include <unordered_map>
using namespace std;

bool isIsomorphic(string s, string t) {
    unordered_map<char, char> m1, m2;
    for (int i = 0; i < s.size(); i++) {
        char cs = s[i], ct = t[i];
        if ((m1.count(cs) && m1[cs] != ct) || (m2.count(ct) && m2[ct] != cs))
            return false;
        m1[cs] = ct;
        m2[ct] = cs;
    }
    return true;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

28. LeetCode: 394 (Decode String)

Question:

Decode nested encoded string like “3[a2[c]]”.

Solution Logic:

Use stacks to hold previous strings and repeat counts.

Code (C++):

```
#include <string>
#include <stack>
using namespace std;

string decodeString(string s) {
    stack<int> counts;
    stack<string> results;
    string current;
    int k = 0;

    for (char c : s) {
        if (isdigit(c)) {
            k = k * 10 + (c - '0');
        } else if (c == '[') {
            counts.push(k);
            results.push(current);
            current = "";
            k = 0;
        } else if (c == ']') {
            int count = counts.top(); counts.pop();
            string prev = results.top(); results.pop();
            string repeated;
            while (count-- > 0) repeated += current;
            current = prev + repeated;
        } else {
            current.push_back(c);
        }
    }
    return current;
}
```

Time Complexity: $O(n * k)$

Space Complexity: $O(n)$

29. LeetCode: 6 (ZigZag Conversion)

Question:

Convert string into zigzag pattern across numRows, then read row by row.

Solution Logic:

Simulate rows, direction change flags, append characters row by row.

Code (C++):

```
#include <string>
#include <vector>
using namespace std;

string convertZigzag(string s, int numRows) {
    if (numRows == 1 || numRows >= s.size()) return s;
    vector<string> rows(min(numRows, int(s.size())));
```

```

int cur = 0;
bool down = false;

for (char c : s) {
    rows[cur].push_back(c);
    if (cur == 0 || cur == numRows - 1) down = !down;
    cur += down ? 1 : -1;
}

string result;
for (string &row : rows) result += row;
return result;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

30. LeetCode: 44 (Wildcard Matching)

Question:

Implement wildcard matching with ? and * (where ? matches one character, * matches zero or more).

Solution Logic:

Use DP / recursion + memo to match string and pattern with wildcards.

Code (C++):

```

#include <string>
#include <vector>
using namespace std;

bool isMatch(string s, string p) {
    int n = s.size(), m = p.size();
    vector<vector<bool>> dp(n + 1, vector<bool>(m + 1, false));
    dp[0][0] = true;

    for (int j = 1; j <= m; j++) {
        if (p[j-1] == '*') dp[0][j] = dp[0][j-1];
    }

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (p[j-1] == s[i-1] || p[j-1] == '?') {
                dp[i][j] = dp[i-1][j-1];
            } else if (p[j-1] == '*') {
                dp[i][j] = dp[i][j-1] || dp[i-1][j];
            } else {
                dp[i][j] = false;
            }
        }
    }

    return dp[n][m];
}

```

Time Complexity: $O(n * m)$

Space Complexity: $O(n * m)$

Great — you're asking for the **Linked List (LL) questions** in C++, in the exact format we've used:

✓Format includes:

- LeetCode number
- Question
- Solution Logic
- C++ Code
- Time & Space Complexity
- Proper numbering continuation (starting from 31)

Linked Lists

31. LeetCode: 206 (Reverse Linked List)

Question:

Reverse a singly linked list.

Solution Logic:

Iteratively reverse the `next` pointers while traversing the list using three pointers: `prev`, `curr`, and `next`.

Code (C++):

```
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};

ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr;
    ListNode* curr = head;
    while (curr) {
        ListNode* nextTemp = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nextTemp;
    }
    return prev;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

32. LeetCode: 141 (Linked List Cycle)

Question:

Detect if a linked list has a cycle.

Solution Logic:

Use Floyd's Cycle Detection (Tortoise and Hare) algorithm — two pointers moving at different speeds.

Code (C++):

```
bool hasCycle(ListNode* head) {
    ListNode* slow = head;
    ListNode* fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

33. LeetCode: 21 (Merge Two Sorted Lists)

Question:

Merge two sorted linked lists and return a new sorted list.

Solution Logic:

Use a dummy node and a tail pointer to build the merged list from two sorted inputs.

Code (C++):

```
ListNode* mergeTwoLists(ListNode* l1, ListNode* l2) {
    ListNode dummy(0);
    ListNode* tail = &dummy;

    while (l1 && l2) {
        if (l1->val < l2->val) {
            tail->next = l1;
            l1 = l1->next;
        } else {
            tail->next = l2;
            l2 = l2->next;
        }
        tail = tail->next;
    }

    tail->next = l1 ? l1 : l2;
    return dummy.next;
}
```

Time Complexity: $O(n + m)$

Space Complexity: $O(1)$

34. LeetCode: 19 (Remove Nth Node From End of List)

Question:

Remove the N-th node from the end of the list.

Solution Logic:

Use two pointers with a gap of n nodes. When the first reaches the end, the second will be just before the node to remove.

Code (C++):

```
ListNode* removeNthFromEnd(ListNode* head, int n) {
    ListNode dummy(0);
    dummy.next = head;
    ListNode* fast = &dummy;
    ListNode* slow = &dummy;

    for (int i = 0; i < n; i++) {
        fast = fast->next;
    }

    while (fast->next) {
        fast = fast->next;
        slow = slow->next;
    }

    slow->next = slow->next->next;
    return dummy.next;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

35. LeetCode: 876 (Middle of the Linked List)

Question:

Return the middle node. If even number of nodes, return second middle.

Solution Logic:

Use slow and fast pointers. Slow moves 1 step, fast moves 2 steps.

Code (C++):

```
ListNode* middleNode(ListNode* head) {
    ListNode* slow = head;
    ListNode* fast = head;

    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }

    return slow;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

36. LeetCode: 203 (Remove Linked List Elements)

Question:

Remove all nodes with a given value.

Solution Logic:

Use a dummy node and skip all nodes that match the given value.

Code (C++):

```
ListNode* removeElements(ListNode* head, int val) {
    ListNode dummy(0);
    dummy.next = head;
    ListNode* current = &dummy;

    while (current->next) {
        if (current->next->val == val)
            current->next = current->next->next;
        else
            current = current->next;
    }

    return dummy.next;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

37. LeetCode: 160 (Intersection of Two Linked Lists)

Question:

Find the node where two linked lists intersect.

Solution Logic:

Traverse both lists; switch heads once you reach end — they will meet at intersection or nullptr.

Code (C++):

```
ListNode* getIntersectionNode(ListNode* headA, ListNode* headB) {
    ListNode* a = headA;
    ListNode* b = headB;

    while (a != b) {
        a = a ? a->next : headB;
        b = b ? b->next : headA;
    }

    return a;
}
```

```
}
```

Time Complexity: $O(n + m)$

Space Complexity: $O(1)$

38. LeetCode: 234 (Palindrome Linked List)

Question:

Check whether a linked list is a palindrome.

Solution Logic:

Find middle, reverse second half, compare both halves.

Code (C++):

```
ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr;
    while (head) {
        ListNode* nextTemp = head->next;
        head->next = prev;
        prev = head;
        head = nextTemp;
    }
    return prev;
}

bool isPalindrome(ListNode* head) {
    if (!head || !head->next) return true;

    ListNode* slow = head;
    ListNode* fast = head;

    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }

    ListNode* secondHalf = reverseList(slow);
    ListNode* firstHalf = head;

    while (secondHalf) {
        if (firstHalf->val != secondHalf->val) return false;
        firstHalf = firstHalf->next;
        secondHalf = secondHalf->next;
    }

    return true;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

39. LeetCode: 2 (Add Two Numbers)

Question:

Add two numbers represented by reversed linked lists.

Solution Logic:

Add digit by digit with carry and create a new list.

Code (C++):

```
ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
    ListNode dummy(0);
    ListNode* curr = &dummy;
    int carry = 0;

    while (l1 || l2 || carry) {
        int val1 = l1 ? l1->val : 0;
        int val2 = l2 ? l2->val : 0;

        int sum = val1 + val2 + carry;
        carry = sum / 10;
        curr->next = new ListNode(sum % 10);
        curr = curr->next;

        if (l1) l1 = l1->next;
        if (l2) l2 = l2->next;
    }

    return dummy.next;
}
```

Time Complexity: $O(\max(n, m))$

Space Complexity: $O(\max(n, m))$

40. LeetCode: 148 (Sort List)

Question:

Sort a linked list in $O(n \log n)$ time and constant space complexity.

Solution Logic:

Use merge sort (split into halves, recursively sort, then merge).

Code (C++):

```
ListNode* merge(ListNode* l1, ListNode* l2) {
    ListNode dummy(0);
    ListNode* tail = &dummy;
    while (l1 && l2) {
        if (l1->val < l2->val) {
            tail->next = l1;
            l1 = l1->next;
        } else {
            tail->next = l2;
            l2 = l2->next;
        }
    }
}
```

```

        tail = tail->next;
    }
    tail->next = 11 ? 11 : 12;
    return dummy.next;
}

```

```

ListNode* sortList(ListNode* head) {
    if (!head || !head->next) return head;

    ListNode* slow = head;
    ListNode

```

- fast = head->next;

```

while (fast && fast->next) {
    slow = slow->next;
    fast = fast->next->next;
}

```

```

ListNode* mid = slow->next;
slow->next = nullptr;

```

```

ListNode* left = sortList(head);
ListNode* right = sortList(mid);

```

```

return merge(left, right);
}

```

Time Complexity: $O(n \log n)$

Space Complexity: $O(\log n)$ (due to recursion)

Stacks

41. LeetCode: 20 (Valid Parentheses)

Question:

Given a string containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid (open brackets closed in correct order).

Solution Logic:

Use a stack to push opening brackets; for each closing, pop and check match.

Code (C++):

```

#include <string>
#include <stack>
using namespace std;

bool isValid(string s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') {

```

```

        st.push(c);
    } else {
        if (st.empty()) return false;
        char top = st.top();
        st.pop();
        if ((c == ')' && top != '(') ||
            (c == ']' && top != '[') ||
            (c == '}' && top != '{'))
            return false;
    }
}
return st.empty();
}

```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

42. LeetCode: 155 (Min Stack)

Question:

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Solution Logic:

Maintain two stacks: one for all values, another for minimum values so far.

Code (C++):

```

#include <stack>
using namespace std;

class MinStack {
    stack<int> st, minSt;
public:
    MinStack() {}

    void push(int val) {
        st.push(val);
        if (minSt.empty() || val <= minSt.top())
            minSt.push(val);
    }
    void pop() {
        if (st.top() == minSt.top())
            minSt.pop();
        st.pop();
    }
    int top() {
        return st.top();
    }
    int getMin() {
        return minSt.top();
    }
};

```

Time Complexity: $O(1)$

Space Complexity: $O(n)$

43. LeetCode: 32 (Longest Valid Parentheses)

Question:

Given a string containing '(' and ')', find length of the longest valid parentheses substring.

Solution Logic:

Use stack of indices, push -1 as base. On encountering ')', pop and compute distance to new top.

Code (C++):

```
#include <string>
#include <stack>
#include <algorithm>
using namespace std;

int longestValidParentheses(string s) {
    stack<int> st;
    st.push(-1);
    int maxLen = 0;
    for (int i = 0; i < s.size(); i++) {
        if (s[i] == '(') {
            st.push(i);
        } else {
            st.pop();
            if (st.empty()) {
                st.push(i);
            } else {
                maxLen = max(maxLen, i - st.top());
            }
        }
    }
    return maxLen;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

44. LeetCode: 225 (Implement Stack using Queues)

Question:

Implement a stack using only two queues.

Solution Logic:

Make push expensive: enqueue new element to second queue, move all from first to second, swap.

Code (C++):

```
#include <queue>
using namespace std;

class MyStack {
    queue<int> q1, q2;
```

```

public:
    MyStack() {}

    void push(int x) {
        q2.push(x);
        while (!q1.empty()) {
            q2.push(q1.front());
            q1.pop();
        }
        swap(q1, q2);
    }

    int pop() {
        int val = q1.front();
        q1.pop();
        return val;
    }

    int top() {
        return q1.front();
    }

    bool empty() {
        return q1.empty();
    }
};

```

Time Complexity: push: $O(n)$, pop/top/empty: $O(1)$

Space Complexity: $O(n)$

45. LeetCode: 232 (Implement Queue using Stacks)

Question:

Implement a queue using two stacks.

Solution Logic:

Use two stacks (s_{in} and s_{out}). Transfer elements from s_{in} to s_{out} when performing pop/peek and s_{out} is empty.

Code (C++):

```

#include <stack>
using namespace std;

class MyQueue {
    stack<int> s_in, s_out;
    void transfer() {
        while (!s_in.empty()) {
            s_out.push(s_in.top());
            s_in.pop();
        }
    }
public:
    MyQueue() {}

    void push(int x) {
        s_in.push(x);
    }

```

```

    }

    int pop() {
        if (s_out.empty()) transfer();
        int val = s_out.top();
        s_out.pop();
        return val;
    }

    int peek() {
        if (s_out.empty()) transfer();
        return s_out.top();
    }

    bool empty() {
        return s_in.empty() && s_out.empty();
    }
};

```

Time Complexity: Amortized $O(1)$

Space Complexity: $O(n)$

46. LeetCode: 394 (Decode String)

Question:

Decode an encoded string with nested patterns like $k[\dots]$.

Solution Logic:

Use stacks: one for counts, one for result strings. On ']', combine top.

Code (C++):

```

#include <string>
#include <stack>
using namespace std;

string decodeString(string s) {
    stack<int> counts;
    stack<string> results;
    string current = "";
    int k = 0;
    for (char c : s) {
        if (isdigit(c)) {
            k = k * 10 + (c - '0');
        } else if (c == '[') {
            counts.push(k);
            results.push(current);
            current = "";
            k = 0;
        } else if (c == ']') {
            int count = counts.top(); counts.pop();
            string prev = results.top(); results.pop();
            string repeated = "";
            for (int i = 0; i < count; i++)
                repeated += current;
            current = prev + repeated;
        } else {

```

```

        current.push_back(c);
    }
}
return current;
}

```

Time Complexity: $O(n * k)$

Space Complexity: $O(n)$

47. LeetCode: 503 (Next Greater Element II)

Question:

In a circular array, find next greater element for each element; if none, return -1.

Solution Logic:

Use stack of indices; loop $2 \times n$ to cover wrap around, assign when greater found.

Code (C++):

```

#include <vector>
#include <stack>
using namespace std;

vector<int> nextGreaterElements(vector<int>& nums) {
    int n = nums.size();
    vector<int> res(n, -1);
    stack<int> st;
    for (int i = 0; i < 2 * n; i++) {
        int val = nums[i % n];
        while (!st.empty() && nums[st.top()] < val) {
            res[st.top()] = val;
            st.pop();
        }
        if (i < n) {
            st.push(i);
        }
    }
    return res;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

48. LeetCode: 856 (Score of Parentheses)

Question:

Compute score of parentheses using rules: $() = 1$, $AB = A+B$, $(A) = 2 \times A$.

Solution Logic:

Use stack to accumulate scores. On $($ push 0, on $)$ pop and compute.

Code (C++):

```

#include <string>
#include <stack>
using namespace std;

int scoreOfParentheses(string S) {
    stack<int> st;
    st.push(0);
    for (char c : S) {
        if (c == '(') {
            st.push(0);
        } else {
            int v = st.top(); st.pop();
            int w = st.top(); st.pop();
            st.push(w + max(2 * v, 1));
        }
    }
    return st.top();
}

```

Time Complexity: O(n)

Space Complexity: O(n)

Queue

49. LeetCode: 933 (Number of Recent Calls)

Question:

Design a recent counter to count number of pings in last 3000 ms.

Solution Logic:

Queue holds timestamps, pop front older than $t - 3000$.

Code (C++):

```

#include <queue>
using namespace std;

class RecentCounter {
    queue<int> q;
public:
    RecentCounter() {}
    int ping(int t) {
        q.push(t);
        while (q.front() < t - 3000) {
            q.pop();
        }
        return q.size();
    }
};

```

Time Complexity: Amortized O(1)

Space Complexity: O(n)

50. LeetCode: 239 (Sliding Window Maximum)

Question:

Given an array and window size k, return maximum for each window.

Solution Logic:

Use deque to keep indices of potential maxima in decreasing order.

Code (C++):

```
#include <vector>
#include <deque>
using namespace std;

vector<int> maxSlidingWindow(vector<int>& nums, int k) {
    deque<int> dq;
    vector<int> res;
    for (int i = 0; i < nums.size(); i++) {
        if (!dq.empty() && dq.front() == i - k) {
            dq.pop_front();
        }
        while (!dq.empty() && nums[dq.back()] < nums[i]) {
            dq.pop_back();
        }
        dq.push_back(i);
        if (i >= k - 1) {
            res.push_back(nums[dq.front()]);
        }
    }
    return res;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(k)$

51. LeetCode: 641 (Design Circular Deque)

Question:

Implement a circular deque with operations insertFront, insertLast, deleteFront, deleteLast, getFront, getRear, isEmpty, isFull.

Solution Logic:

Use fixed-size array, pointers for front, rear, and count.

Code (C++):

```
#include <vector>
using namespace std;

class MyCircularDeque {
    vector<int> dq;
    int frontIdx, rearIdx, capacity, count;
public:
    MyCircularDeque(int k) {
        capacity = k;
```

```

        dq.resize(k);
        frontIdx = 0;
        rearIdx = 0;
        count = 0;
    }
    bool insertFront(int value) {
        if (isFull()) return false;
        frontIdx = (frontIdx - 1 + capacity) % capacity;
        dq[frontIdx] = value;
        count++;
        return true;
    }
    bool insertLast(int value) {
        if (isFull()) return false;
        dq[rearIdx] = value;
        rearIdx = (rearIdx + 1) % capacity;
        count++;
        return true;
    }
    bool deleteFront() {
        if (isEmpty()) return false;
        frontIdx = (frontIdx + 1) % capacity;
        count--;
        return true;
    }
    bool deleteLast() {
        if (isEmpty()) return false;
        rearIdx = (rearIdx - 1 + capacity) % capacity;
        count--;
        return true;
    }
    int getFront() {
        if (isEmpty()) return -1;
        return dq[frontIdx];
    }
    int getRear() {
        if (isEmpty()) return -1;
        return dq[(rearIdx - 1 + capacity) % capacity];
    }
    bool isEmpty() {
        return count == 0;
    }
    bool isFull() {
        return count == capacity;
    }
};

```

Time Complexity: O(1)

Space Complexity: O(k)

52. LeetCode: 622 (Design Circular Queue)

Question:

Implement a circular queue with enqueue, dequeue, Front, Rear, isEmpty, isFull operations.

Solution Logic:

Use fixed buffer with front, rear pointers and count for capacity.

Code (C++):

```
#include <vector>
using namespace std;

class MyCircularQueue {
    vector<int> que;
    int frontIdx, rearIdx, capacity, count;
public:
    MyCircularQueue(int k) {
        capacity = k;
        que.resize(k);
        frontIdx = 0;
        rearIdx = 0;
        count = 0;
    }
    bool enQueue(int value) {
        if (isFull()) return false;
        que[rearIdx] = value;
        rearIdx = (rearIdx + 1) % capacity;
        count++;
        return true;
    }
    bool deQueue() {
        if (isEmpty()) return false;
        frontIdx = (frontIdx + 1) % capacity;
        count--;
        return true;
    }
    int Front() {
        if (isEmpty()) return -1;
        return que[frontIdx];
    }
    int Rear() {
        if (isEmpty()) return -1;
        return que[(rearIdx - 1 + capacity) % capacity];
    }
    bool isEmpty() {
        return count == 0;
    }
    bool isFull() {
        return count == capacity;
    }
};
```

Time Complexity: O(1)

Space Complexity: O(k)

53. LeetCode: 346 (Moving Average from Data Stream)

Question:

Design a class to compute moving average of data stream with fixed window size.

Solution Logic:

Keep queue of last k values and running sum.

Code (C++):

```
#include <queue>
```

```
using namespace std;

class MovingAverage {
    queue<int> q;
    int size;
    double sum;
public:
    MovingAverage(int k) {
        size = k;
        sum = 0;
    }
    double next(int val) {
        q.push(val);
        sum += val;
        if (q.size() > size) {
            sum -= q.front();
            q.pop();
        }
        return sum / q.size();
    }
};
```

Time Complexity: $O(1)$

Space Complexity: $O(k)$

Trees and Binary Search Trees

54. LeetCode: 104 (Maximum Depth of Binary Tree)

Question:

Given the root of a binary tree, return its maximum depth.

Solution Logic:

Use DFS (recursion) to find max depth of left and right subtree, return $1 + \max$ of both.

Code (C++):

```
struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

int maxDepth(TreeNode* root) {
    if (!root) return 0;
    return 1 + max(maxDepth(root->left), maxDepth(root->right));
}
```

Time Complexity: $O(n)$

Space Complexity: $O(h)$ — height of the tree (recursion stack)

55. LeetCode: 101 (Symmetric Tree)

Question:

Check whether a binary tree is symmetric around its center (mirror image).

Solution Logic:

Use recursion to compare left and right subtrees for mirror symmetry.

Code (C++):

```
bool isMirror(TreeNode* t1, TreeNode* t2) {
    if (!t1 && !t2) return true;
    if (!t1 || !t2) return false;
    return (t1->val == t2->val)
        && isMirror(t1->left, t2->right)
        && isMirror(t1->right, t2->left);
}

bool isSymmetric(TreeNode* root) {
    return isMirror(root, root);
}
```

Time Complexity: $O(n)$

Space Complexity: $O(h)$

56. LeetCode: 98 (Validate Binary Search Tree)

Question:

Given a binary tree, determine if it is a valid BST.

Solution Logic:

Use recursion and maintain valid min/max ranges for nodes.

Code (C++):

```
bool validate(TreeNode* root, long long minVal, long long maxVal) {
    if (!root) return true;
    if (root->val <= minVal || root->val >= maxVal) return false;
    return validate(root->left, minVal, root->val) && validate(root->right, root->val,
maxVal);
}

bool isValidBST(TreeNode* root) {
    return validate(root, LONG_MIN, LONG_MAX);
}
```

Time Complexity: $O(n)$

Space Complexity: $O(h)$

57. LeetCode: 236 (Lowest Common Ancestor of a Binary Tree)

Question:

Find the lowest common ancestor (LCA) of two given nodes in a binary tree.

Solution Logic:

Recursively search left and right; if both sides return non-null, current node is LCA.

Code (C++):

```
TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
    if (!root || root == p || root == q) return root;
    TreeNode* left = lowestCommonAncestor(root->left, p, q);
    TreeNode* right = lowestCommonAncestor(root->right, p, q);
    if (left && right) return root;
    return left ? left : right;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(h)$

58. LeetCode: 700 (Search in a Binary Search Tree)

Question:

Search a value in a BST and return the subtree rooted with that node.

Solution Logic:

Use BST property: traverse left/right accordingly.

Code (C++):

```
TreeNode* searchBST(TreeNode* root, int val) {
    if (!root || root->val == val) return root;
    if (val < root->val)
        return searchBST(root->left, val);
    else
        return searchBST(root->right, val);
}
```

Time Complexity: $O(h)$

Space Complexity: $O(h)$

59. LeetCode: 701 (Insert into a Binary Search Tree)

Question:

Insert a value into BST and return the root node of the updated tree.

Solution Logic:

Traverse BST, find correct position and insert node.

Code (C++):

```
TreeNode* insertIntoBST(TreeNode* root, int val) {
    if (!root) return new TreeNode(val);
    if (val < root->val)
```

```

        root->left = insertIntoBST(root->left, val);
    else
        root->right = insertIntoBST(root->right, val);
    return root;
}

```

Time Complexity: $O(h)$

Space Complexity: $O(h)$

60. LeetCode: 450 (Delete Node in a BST)

Question:

Delete a node with a given key in BST and return the root.

Solution Logic:

Find node, handle three cases: leaf, one child, two children (replace with inorder successor).

Code (C++):

```

TreeNode* findMin(TreeNode* root) {
    while (root->left) root = root->left;
    return root;
}

TreeNode* deleteNode(TreeNode* root, int key) {
    if (!root) return nullptr;
    if (key < root->val) root->left = deleteNode(root->left, key);
    else if (key > root->val) root->right = deleteNode(root->right, key);
    else {
        if (!root->left) return root->right;
        else if (!root->right) return root->left;
        TreeNode* minNode = findMin(root->right);
        root->val = minNode->val;
        root->right = deleteNode(root->right, minNode->val);
    }
    return root;
}

```

Time Complexity: $O(h)$

Space Complexity: $O(h)$

Bonus: Advanced / Mixed Questions

61. LeetCode: 297 (Serialize and Deserialize Binary Tree)

Question:

Design an algorithm to serialize and deserialize a binary tree.

Solution Logic:

Use preorder traversal with markers for null nodes.

Code (C++):

```
class Codec {
public:
    void serializeHelper(TreeNode* root, ostream &out) {
        if (!root) {
            out << "# ";
            return;
        }
        out << root->val << " ";
        serializeHelper(root->left, out);
        serializeHelper(root->right, out);
    }

    string serialize(TreeNode* root) {
        ostream out;
        serializeHelper(root, out);
        return out.str();
    }

    TreeNode* deserializeHelper(istream &in) {
        string val;
        in >> val;
        if (val == "#") return nullptr;
        TreeNode* node = new TreeNode(stoi(val));
        node->left = deserializeHelper(in);
        node->right = deserializeHelper(in);
        return node;
    }

    TreeNode* deserialize(string data) {
        istream in(data);
        return deserializeHelper(in);
    }
};
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

62. LeetCode: 329 (Longest Increasing Path in a Matrix)

Question:

Find the length of the longest increasing path in a matrix.

Solution Logic:

Use DFS + memoization to find max increasing paths from each cell.

Code (C++):

```
class Solution {
    int rows, cols;
    vector<vector<int>> directions {{1,0},{-1,0},{0,1},{0,-1}};
    vector<vector<int>> memo;
public:
    int dfs(int r, int c, vector<vector<int>>& matrix) {
        if (memo[r][c]) return memo[r][c];
        int maxLength = 1;
```

```

        for (auto& dir : directions) {
            int nr = r + dir[0], nc = c + dir[1];
            if (nr >= 0 && nr < rows && nc >= 0 && nc < cols && matrix[nr][nc] >
matrix[r][c]) {
                maxLength = max(maxLength, 1 + dfs(nr, nc, matrix));
            }
        }
        return memo[r][c] = maxLength;
    }

    int longestIncreasingPath(vector<vector<int>>& matrix) {
        if (matrix.empty()) return 0;
        rows = matrix.size();
        cols = matrix[0].size();
        memo.assign(rows, vector<int>(cols, 0));
        int maxPath = 0;
        for (int r = 0; r < rows; r++) {
            for (int c = 0; c < cols; c++) {
                maxPath = max(maxPath, dfs(r, c, matrix));
            }
        }
        return maxPath;
    }
};

```

Time Complexity: $O(mn)$

Space Complexity: $O(mn)$

63. LeetCode: 207 (Course Schedule)

Question:

Given a number of courses and prerequisites, determine if you can finish all courses.

Solution Logic:

Detect cycle in directed graph using DFS.

Code (C++):

```

class Solution {
    vector<vector<int>> graph;
    vector<int> visited; // 0=unvisited, 1=visiting, 2=visited

    bool dfs(int node) {
        if (visited[node] == 1) return true; // cycle
        if (visited[node] == 2) return false;

        visited[node] = 1;
        for (int neighbor : graph[node]) {
            if (dfs(neighbor)) return true;
        }
        visited[node] = 2;
        return false;
    }

public:
    bool canFinish(int numCourses, vector<vector<int>>& prerequisites) {
        graph.resize(numCourses);
    }
};

```

```

        visited.assign(numCourses, 0);

        for (auto &pre : prerequisites) {
            graph[pre[1]].push_back(pre[0]);
        }

        for (int i = 0; i < numCourses; i++) {
            if (dfs(i)) return false;
        }
        return true;
    }
};

```

Time Complexity: $O(V + E)$

Space Complexity: $O(V + E)$

64. LeetCode: 208 (Implement Trie (Prefix Tree))

Question:

Implement a trie with insert, search, and startsWith methods.

Solution Logic:

Use a tree structure where each node contains links to child nodes and a flag for word end.

Code (C++):

```

class TrieNode {
public:
    TrieNode* children[26];
    bool isEnd;
    TrieNode() {
        isEnd = false;
        for (int i = 0; i < 26; i++) children[i] = nullptr;
    }
};

class Trie {
    TrieNode* root;
public:
    Trie() { root = new TrieNode(); }

    void insert(string word) {
        TrieNode* node = root;
        for (char c : word) {
            int idx = c - 'a';
            if (!node->children[idx]) node->children[idx] = new TrieNode();
            node = node->children[idx];
        }
        node->isEnd = true;
    }

    bool search(string word) {
        TrieNode* node = root;
        for (char c : word) {
            int idx = c - 'a';
            if (!node->children[idx]) return false;
            node = node->children[idx];
        }
    }
};

```

```

    }
    return node->isEnd;
}

bool startsWith(string prefix) {
    TrieNode* node = root;
    for (char c : prefix) {
        int idx = c - 'a';
        if (!node->children[idx]) return false;
        node = node->children[idx];
    }
    return true;
}
};

```

Time Complexity: $O(m)$ per operation, where m is the length of the word/prefix

Space Complexity: $O(n * m)$ for n words of average length m

65. LeetCode: 253 (Meeting Rooms II)

Question:

Given meeting time intervals, find the minimum number of conference rooms required.

Solution Logic:

Use a min-heap to track ongoing meetings and allocate rooms.

Code (C++):

```

class Solution {
public:
    int minMeetingRooms(vector<vector<int>>& intervals) {
        if (intervals.empty()) return 0;
        sort(intervals.begin(), intervals.end(), [](auto &a, auto &b){ return a[0] <
b[0]; });

        priority_queue<int, vector<int>, greater<int>> pq;
        pq.push(intervals[0][1]);

        for (int i = 1; i < intervals.size(); i++) {
            if (intervals[i][0] >= pq.top()) pq.pop();
            pq.push(intervals[i][1]);
        }
        return pq.size();
    }
};

```

Time Complexity: $O(n \log n)$

Space Complexity: $O(n)$

66. LeetCode: 752 (Open the Lock)

Question:

Minimum moves to open a lock with deadends and target using BFS.

Solution Logic:

BFS through possible states avoiding deadends.

Code (C++):

```
class Solution {
public:
    int openLock(vector<string>& deadends, string target) {
        unordered_set<string> dead(deadends.begin(), deadends.end());
        if (dead.count("0000")) return -1;

        queue<string> q;
        unordered_set<string> visited;
        q.push("0000");
        visited.insert("0000");

        int steps = 0;
        while (!q.empty()) {
            int size = q.size();
            while (size--) {
                string curr = q.front(); q.pop();
                if (curr == target) return steps;
                if (dead.count(curr)) continue;

                for (int i = 0; i < 4; i++) {
                    string up = curr, down = curr;
                    up[i] = (curr[i] == '9') ? '0' : curr[i] + 1;
                    down[i] = (curr[i] == '0') ? '9' : curr[i] - 1;

                    if (!visited.count(up)) {
                        visited.insert(up);
                        q.push(up);
                    }
                    if (!visited.count(down)) {
                        visited.insert(down);
                        q.push(down);
                    }
                }
            }
            steps++;
        }
        return -1;
    }
};
```

Time Complexity: $O(10^4)$ (max states)

Space Complexity: $O(10^4)$

67. LeetCode: 139 (Word Break)**Question:**

Given a string and dictionary, determine if string can be segmented into dictionary words.

Solution Logic:

Use DP to check substring segmentation.

Code (C++):

```
class Solution {
public:
    bool wordBreak(string s, vector<string>& wordDict) {
        unordered_set<string> dict(wordDict.begin(), wordDict.end());
        vector<bool> dp(s.size() + 1, false);
        dp[0] = true;

        for (int i = 1; i <= s.size(); i++) {
            for (int j = 0; j < i; j++) {
                if (dp[j] && dict.count(s.substr(j, i - j))) {
                    dp[i] = true;
                    break;
                }
            }
        }
        return dp[s.size()];
    }
};
```

Time Complexity: $O(n^2)$

Space Complexity: $O(n)$

68. LeetCode: 207 (Course Schedule) (already given, skip here)

69. LeetCode: 200 (Number of Islands)

Question:

Count the number of islands (connected groups of '1's) in a grid.

Solution Logic:

Use DFS or BFS to traverse and mark visited.

Code (C++):

```
class Solution {
public:
    void dfs(vector<vector<char>>& grid, int r, int c) {
        int rows = grid.size(), cols = grid[0].size();
        if (r < 0 || c < 0 || r >= rows || c >= cols || grid[r][c] == '0') return;
        grid[r][c] = '0'; // mark visited

        dfs(grid, r+1, c);
        dfs(grid, r-1, c);
        dfs(grid, r, c+1);
        dfs(grid, r, c-1);
    }

    int numIslands(vector<vector<char>>& grid) {
        int count = 0;
        for (int r = 0; r < grid.size(); r++) {
            for (int c = 0; c < grid[0].size(); c++) {
```

```

        if (grid[r][c] == '1') {
            count++;
            dfs(grid, r, c);
        }
    }
}
return count;
}
};

```

Time Complexity: $O(mn)$

Space Complexity: $O(mn)$

70. LeetCode: 417 (Pacific Atlantic Water Flow)

Question:

Find cells where water can flow to both Pacific and Atlantic oceans on a height matrix.

Solution Logic:

Use two BFS/DFS from oceans and find intersections.

Code (C++):

```

class Solution {
    vector<vector<int>> directions{{1,0},{-1,0},{0,1},{0,-1}};
    void dfs(vector<vector<int>>& matrix, vector<vector<bool>>& visited, int r, int c) {
        visited[r][c] = true;
        for (auto& d : directions) {
            int nr = r + d[0], nc = c + d[1];
            if (nr >= 0 && nr < matrix.size() && nc >= 0 && nc < matrix[0].size() &&
                !visited[nr][nc] && matrix[nr][nc] >= matrix[r][c]) {
                dfs(matrix, visited, nr, nc);
            }
        }
    }
public:
    vector<vector<int>> pacificAtlantic(vector<vector<int>>& matrix) {
        if (matrix.empty()) return {};
        int m = matrix.size(), n = matrix[0].size();
        vector<vector<bool>> pacific(m, vector<bool>(n, false));
        vector<vector<bool>> atlantic(m, vector<bool>(n, false));

        for (int i = 0; i < m; i++) {
            dfs(matrix, pacific, i, 0);
            dfs(matrix, atlantic, i, n - 1);
        }
        for (int j = 0; j < n; j++) {
            dfs(matrix, pacific, 0, j);
            dfs(matrix, atlantic, m - 1, j);
        }

        vector<vector<int>> result;
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (pacific[i][j] && atlantic[i][j]) result.push_back({i, j});
            }
        }
    }
}

```

```
        return result;
    }
};
```

Time Complexity: $O(mn)$

Space Complexity: $O(mn)$

Congratulations!

You have successfully completed this DSA & Tech guide. This marks a step forward in your coding journey! Remember, learning never stops—keep practicing, solving problems, and building projects. Every line of code you write brings you closer to becoming a master of Data Structures, Algorithms, and modern tech skills.

Stay connected with us for more coding tutorials, challenges, tips, and tech content on Instagram: [@SYNTAXERROR](#). Let's grow together as a community of passionate coders!

© 2025-26 SYNTAX ERROR. All Rights Reserved.

This content is exclusively created by SYNTAX ERROR. Unauthorized copying, sharing, or redistribution of this DSA & Tech content is strictly prohibited. Protecting intellectual property ensures quality learning for everyone and maintains the integrity of this work.